University at Buffalo

Project 5

Competitive learning based image compressor

1. Download `barbara.png` from project 5 folder on UB-Learns. Parse it to a variable. The result should be a 512×512 matrix, call it A, where each entry of the matrix A ranges from 0 to 255, where 0 represents a black pixel and 255 represents a white pixel. Correct parsing is important as otherwise your error calculations will be wrong.

   Q1: How many bits are needed to represent the (uncompressed) `barbara.png` image as described above? Justify your answer.

2. Consider representing A as

$$A = \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1n} \\ A_{21} & A_{22} & \cdots & A_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & \cdots & A_{nn} \end{bmatrix}$$

   where each A_{ij} is a $d \times d$ matrix, d divides 512, and

$$n = \frac{512}{d}.$$

   The n^2 submatrices A_{ij}, $i, j \in \{1, \ldots, n\}$, each of dimensions $d \times d$, will be your training set.

3. Initialize neuron weights $X_1, \ldots, X_K$ represented as $d \times d$ matrices arbitrarily (e.g., each entry can be chosen uniformly at random on $[0, 255]$). Use learning rate of 0.5 in your design.

4. Train the weights using competitive learning. Given that the input to the neural network is A_{ij} for some $i, j \in \{1, \ldots, n\}$, the winning neuron should be chosen as the neuron whose weights are closest to A_{ij}:

$$k^* = \arg\min_k \|A_{ij} - X_k\|^2,$$

   where $\|\cdot\|^2$ represents the sum of squares of the entries of a matrix.

Only the winning neuron's weights should be updated by using the update below:

$$X_{k^*} \leftarrow (1-\eta)X_{k^*} + \eta A_{ij},$$

where η is the learning rate.

5. For the competitive learning algorithm discussed in class, the winning neuron is the one that maximizes the inner product. Here, the winning neuron is the one that minimizes the Euclidean distance, while the weight update method remains the same. Do the neuron weights converge under this modified competitive learning algorithm? Mathematically prove your claim.

6. Suppose many epochs of training produce weights $X_1, \ldots, X_K$. During testing, find the winning neuron for each subblock:

$$k^*(i,j) = \arg\min_k \|A_{ij} - X_k\|^2, \qquad i,j \in \{1,\ldots,n\}.$$

No weights are updated during testing. Define

$$A' = \begin{bmatrix} X_{k^*(1,1)} & X_{k^*(1,2)} & \cdots & X_{k^*(1,n)} \\ X_{k^*(2,1)} & X_{k^*(2,2)} & \cdots & X_{k^*(2,n)} \\ \vdots & \vdots & \ddots & \vdots \\ X_{k^*(n,1)} & X_{k^*(n,2)} & \cdots & X_{k^*(n,n)} \end{bmatrix}.$$

The root mean squared error (RMS) is

$$\frac{1}{512}\|A' - A\|.$$

Q2: How many bits are needed to represent A'? Justify your answer.

7. Nmae your file `Proj5`, then upload your well commented code into UB-Learns.

The code should:

- Generate A' for the case $d = 4$ and $K = 8$.
- Display it as a grayscale image.
- Print the resulting RMS after each epoch.
- Print nothing else.
- Execute without modification and finish within 3 minutes.
- Assume `barbara.png` is in the same directory as the code.
- Be general enough so that d and K can be changed.

In the written report, include:

- Final images corresponding to $d = 4$ and $K \in \{2, 8, 32\}$.

- Final RMS values for those cases.
- Epoch number versus RMS graphs.

Hint: Some neurons may never become winners because of poor initialization. Reinitialize such neurons (e.g., set their weights equal to one of the training inputs) to improve RMS performance.

Q3: Explain how and why competitive learning achieves compression while remaining faithful to the original image.